CSF205:
Web Service Development

Week 1: Introduction

Swansea University Prifysgol Abertawe
Department of Computer Science
Applied Software Engineering

Introducing: Me

Hi.

I'm **Filippos.**

- You had the misfortune of having me for 2 terms already
  - ***Life is unkind. Better luck next term!***



Introducing: My Teaching

Lectures:

- I talk.
- I talk a lot.
- I ask questions.
- I'm happy to be interrupted.
  - But please put your hand up…
- I'm trying new things each year
  - Tell me what things I do work or don't work for you!

Canvas:

- Content will be up a week in advance
- Slides may change.
- Changes will be "contextual" & aesthetic.
- Sometimes there will be supplementary notes.

NOTICE!



Look out for this on slides!
It marks the content as contextual and **NOT** examinable content!



DISCLAIMER

Let P represent *"The web is dense and full of horrors."*

Let Q represent *"This module is about web technologies."*

Let R represent *"This module is dense and full of horrors."*

Thus:

$(P \wedge Q) \rightarrow R$





Module Overview: Composite Topics (that fit on one slide)

The old (verbose) title of this module paints a clearer picture

Data Representation, Markup Languages & Web Services

Data types	Hierarchies	Distributed Programming
---	---	---
Data Structures	New Syntax	Concurrency
---	---	---
Serialisation	Metadata	Interfaces
---		---
Encoding		TCP / IP / HTTP

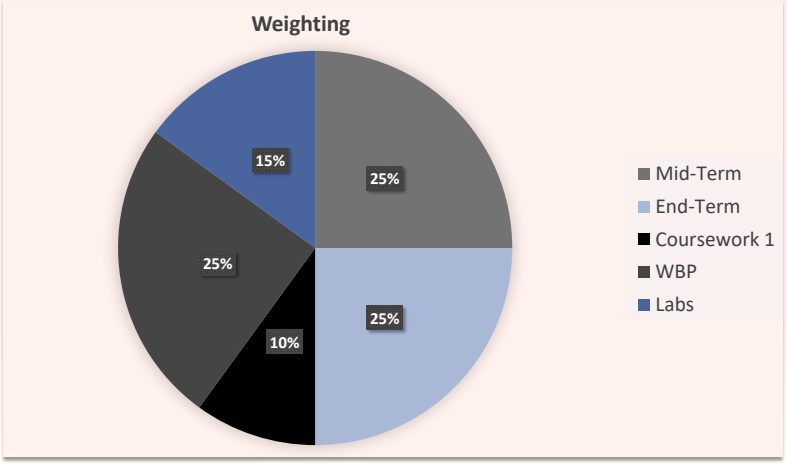
Module Overview: What this is not

A module on:

- Becoming a Java Wizard
- Building APIs
- Learning the Web Stack
- Networking
- Back-end development
- Masochism

But you will learn elements of each . . .

Module Overview: Assessments



Learning Outcomes

Lecture

- Foundations of computer networks
- Sending data via the "TCP/IP" stack
- Solving problems with distributed computing
- Primitive data handling with network sockets

Lab

- Writing your first distributed program in Java
- Understanding how network sockets work at a high level
- Designing a 'protocol' for a simple request-response

PART 0

Foundations of binary representation



Binary Representation

- Computers represent data as 0's and 1s'. We call those **bits.**
- Any and all data types are represented as bits.
- Let's see some examples of integers.

Integer	Binary representation
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000
9	1001
10	1010

Binary Representation

- We sometimes allow ourselves a "fixed" size binary **word** to store data
- If we say "I have a 32-bit word" it means we have 32 0's and 1's.
- Using those bits we can represent all positive integers from 0 to 4,294,967,296:

0 = 00000000 00000000 00000000 00000000

4,294,967,295 = 11111111 11111111 11111111 11111111

- With an 8-bit word, we can represent positive integers from 0 to 255

0 = 00000000

255 = 11111111

Binary Representation

Rule of thumb: You can convert any decimal integer to binary and vice-versa using the bucket technique.

...	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
...	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	1

Powers of 2

Bits

To convert binary **B** to decimal:

- Copy the bits of **B** into the buckets
- Sum up the decimals that have a 1 underneath them

To convert decimal **X** to binary:

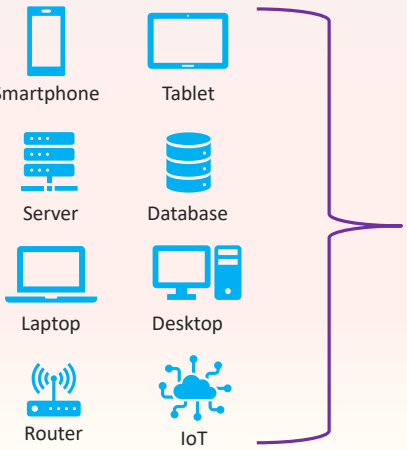
- Start from the left. For every column **C**
- Ask “does **C** go into **X**?”
- If yes, put 1 in the box, and set **X = X-C**
- If no, put 0 in the box

PART 1

Introduction, Computer Networks and Distributed Systems

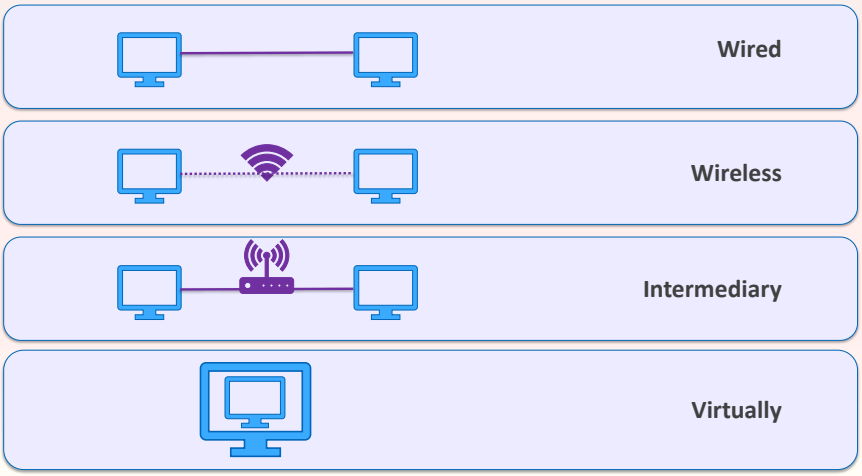


Terminology: Entities



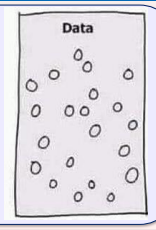
- Machine
- Computer
- Device
- Host
- Node
- System
- Client / Server

Terminology: Connection

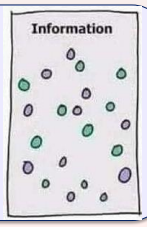


Terminology: Data & Information

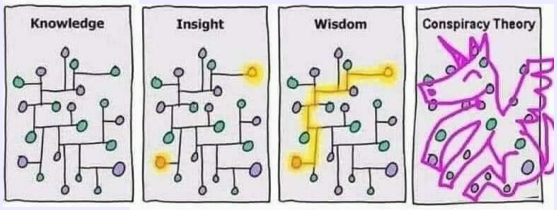
Recorded Observations.



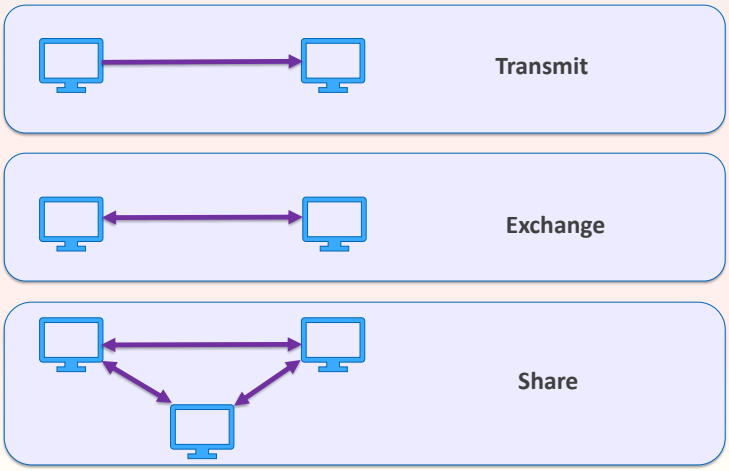
Data with **Purpose**.



Data **Applied** to that Purpose

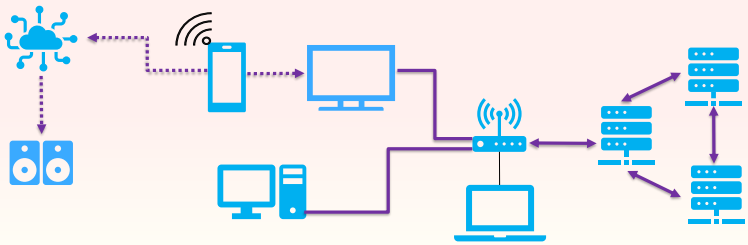


Terminology: Communication

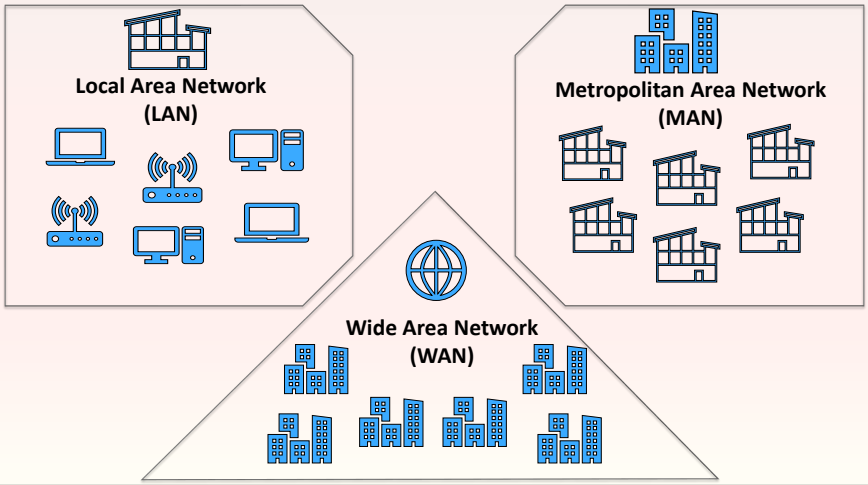


Computer Networks: A Definition

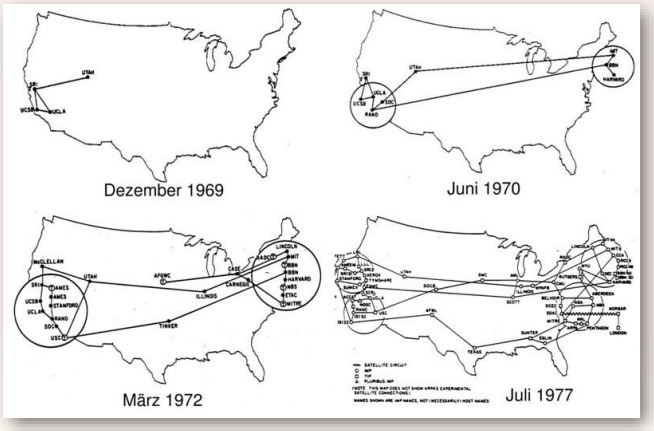
- A collection of **autonomous** computers.
- **Connected** using a guided or unguided medium.
- With the **purpose** of **communicating data** and/or **resources**.



Computer Networks: Classifications



Computer Networks: ARPANet



(Image Source: https://flic.kr/p/GpB7Hw)

Concept Check: Network Communication

How does data move from one device to another?

Layers of abstraction!



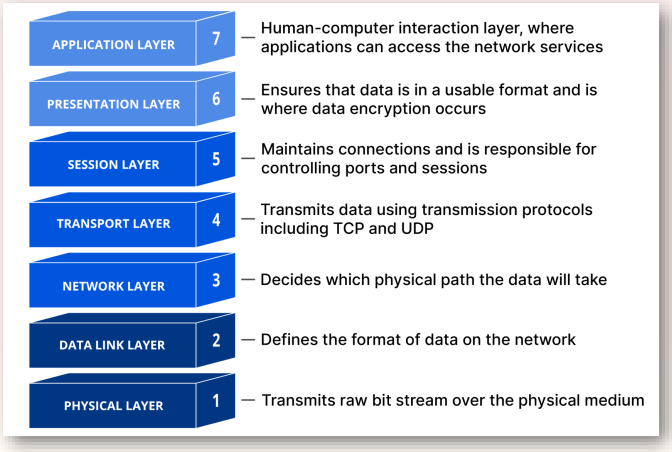
Network Architectures: The OSI Model

Abstracted layers of network comms.

Useful for learning and design.

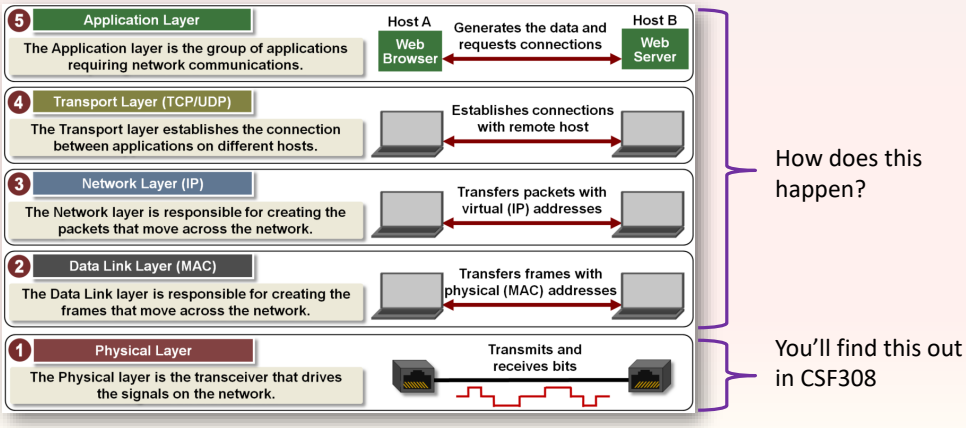
Can help with troubleshooting in IT.

Largely theoretical.



https://cf-assets.www.cloudflare.com/slt3lc6tev37/62H2Etm3LIFHTgmkjLmlxp/59ff240fb3ebdc7794ffaa6e1d69b7c2/osi_model_7_layers.png

Network Architectures: TCP/IP



How does this happen?

You'll find this out in CSF308

TCP/IP: Addressing – Data Link

MAC address (Media Access Control address):

- Globally unique, 48-bit number, identifying a network interface card (NIC) on a specific network.
- Assigned by IEEE and hardware vendors. (Although they can “change”)
- Often shown in hexadecimal format.
- Ensures ethernet frames on a network are delivered to correct devices.
- Facilitates assignment of IP Addresses.
- Devices use MAC addresses to route frames between each other on the same physical or logical network (like a LAN).

TCP/IP: Recognise the MAC

Recognise this?

MAC Address: f8:4d:89:ff:26:01

Apple, Inc



TCP/IP: The IP Address

Each network interface has an IP address:
(IPv4): 32-bit number, range: 0 to 4,294,967,295

Example: `10001001001011000000001000000001`

`10001001.00101100.00000010.00000001`

`137.44.2.1`

Special cases

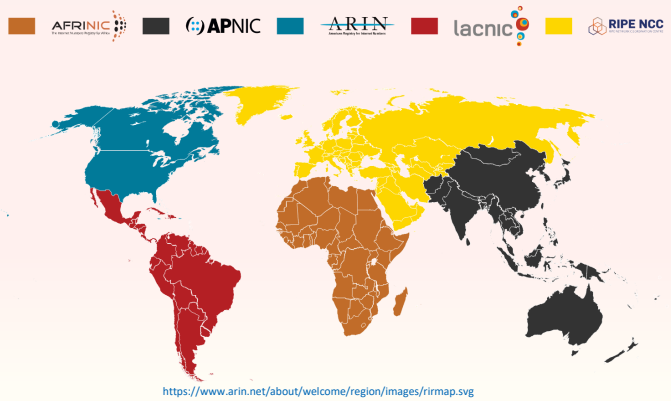
- All zeros: “this network” or “unspecified"
- All ones: “broadcast”
- `127.0.0.1` – loopback: keep it local but pretend it’s an incoming packet.

TCP/IP: Dynamic Host Configuration Protocol

- Mostly plays a role within LANs - but applies globally.
- Dynamically maps MAC address to IP addresses.
- Managed by a DHCP server (typically your router in a home network)
- Devices, when connected to the network, seek out the DHCP server.
- The server responds with configuration information and an available IP address.
- Nationally, your router acts as the client, communicating to your ISP's DHCP server to get its 'public' IP address.

TCP/IP: Addressing – Network

The Internet Assigned Numbers Authority (IANA) coordinates allocation of global IP addresses to Regional Internet Registries (RIRs).



https://www.arin.net/about/welcome/region/images/rirmap.svg

TCP/IP: Domain Name System

Remembering numeric IP addresses can get complicated.

We prefer names like

- https://www.swansea.ac.uk

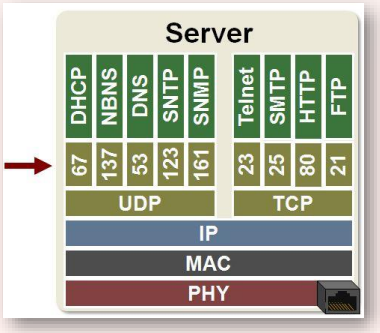
As opposed to

- 137.44.60.170

The Domain Name System (DNS) protocol is responsible for this mapping.

- Hierarchical database that stores domain names and corresponding IPs
- DNS Servers store these databases, starting locally and grow globally.
- Servers unable to resolve a domain name forward requests "up" the chain of DNS servers.
- Devices and DNS servers will also cache recent resolutions, meaning they can provide results faster – also can become outdated.

TCP/IP: Ports



- Applications are managed by processes.
- Ports identify processes on a host.
- They are virtual addresses.
- Common applications have designated ports.
- Ephemeral ports are free to use by Clients.

Note: Never use a port < 1024 unless you mean to use a specific service!

Ports are 16-bit numbers
Range: 0-65525

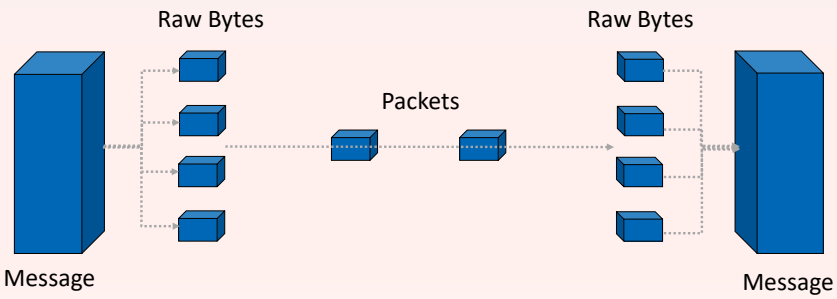
Concept Check: Sending Data

MAC, TCP/IP, Ports, DCHP and DNS provide device-device addressing.

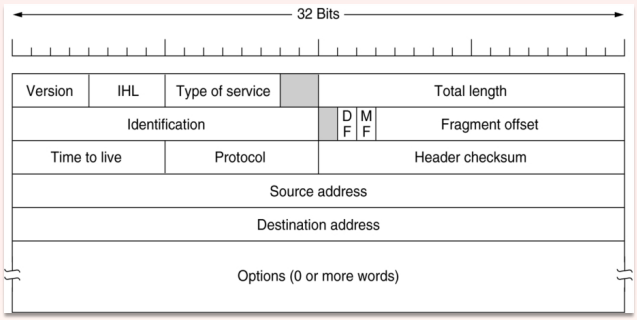
What does the data look like?



TCP/IP: Sending Data



TCP/IP: The IP Packet



TCP/IP doesn't send a stream of data.

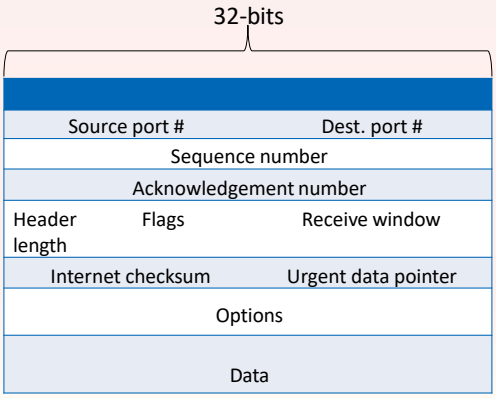
It emulates it by sending chunks of data.

These chunks are called packets.

As far as most programmers are concerned, it is sent and received as a stream.

TCP/IP: Transmission Control Protocol

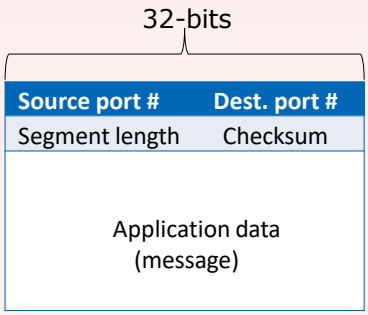
- Stateful, reliable, connection-oriented.
- Full duplex service.
- 3-way handshake to establish connection.
- TCP code can buffer bytes, rather than sending immediately.



TCP/IP: User Datagram Protocol

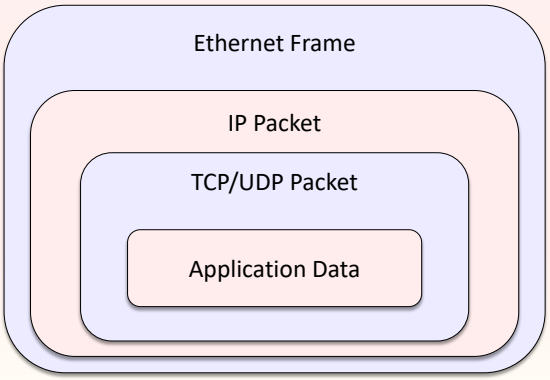
A thin layer above IP

- Sends data to a net socket
- No guarantees.
- No state maintenance or connection setup etc.
- Popular where data can be lost without breaking an application.



TCP/IP: The Simple Picture

You can think of data sent over a network as a nested envelope:



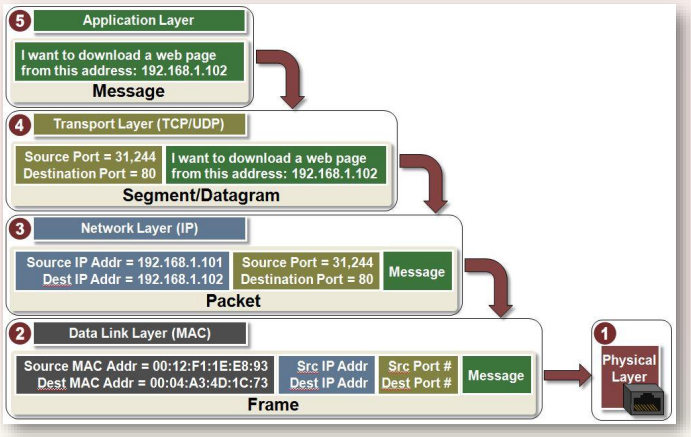
TCP/IP: Software Model

Layer #	Layer Name	Protocol	Protocol Data Unit	Addressing
5	Application	HTTP, SMTP, etc…	Messages	n/a
4	Transport	TCP/UDP	Segments/ Datagrams	Port #s
3	Network or Internet	IP	Packets	IP Address
2	Data Link	Ethernet, Wi-Fi	Frames	MAC Address
1	Physical	10 Base T, 802.11	Bits	n/a

Network protocols are implemented in **software**.

Computer systems implement network protocols as part of their OS (operating system).

TCP/IP: Sample Message



TCP/IP: So, what?

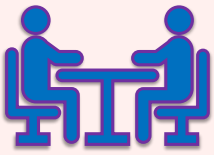
How does this apply to us?

- TCP/IP defines where “things” live in the network stack.
- It also explains how data is sent and how addressing is works.
- It introduces many core concepts and theory that developers of web-based applications need to be aware of.
- It lays the foundation for pretty much all other networking protocols.



Summary: Computer Networking and the Web

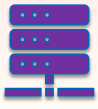
- Data passes through and between computers
- Data is represented and exchanged through layers
- Computers networks are held together by those layers and their protocols
- Layers and protocols form architectures
- As far as we’re concerned, the TCP/IP architecture is the most important.
- The Web, and other technologies we go on to study, are built on top of these models.



Next Lecture: Distributed Computing

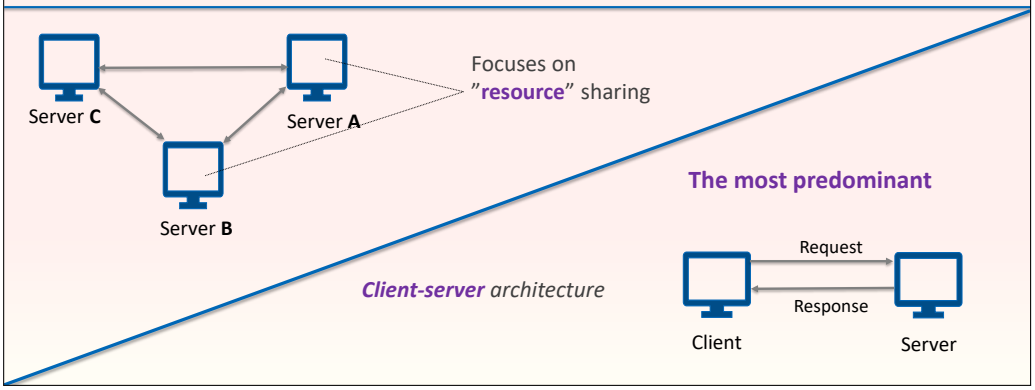
PART 2

Distributed Programming and Sockets

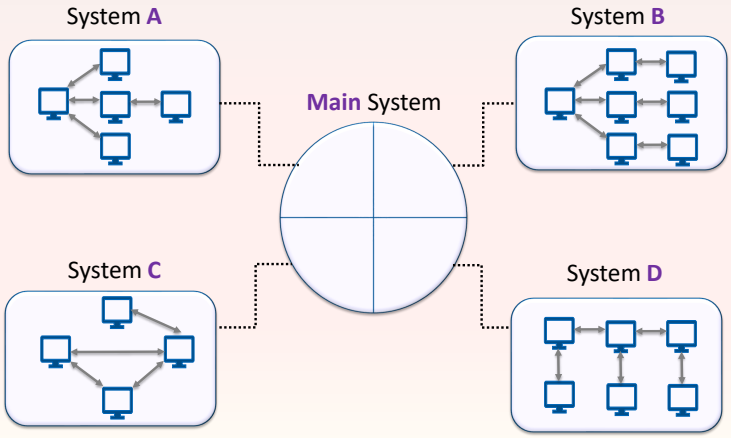


Distributed Computing: What is it?

Multiple hardware or software components communicating and coordinate their actions across a network by passing a **message**



Distributed Computing: Functional Separation



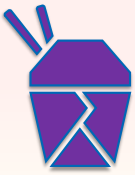
Distributed Computing: The Key Takeaway

Distributed programming:

- Is about coordinated problem solving.
- Constructing and exchanging messages.
- Can get messy without consensus and information.

The key takeaways of this module:

- How and why things get messy.
- How standardisation and design helps prevent it.



Sockets: Not the kind you would stick a fork in

Sockets are a communication endpoint for sending data over a network.

They've been around since the 1970s.

Since then, they have become a fundamental networking primitive.

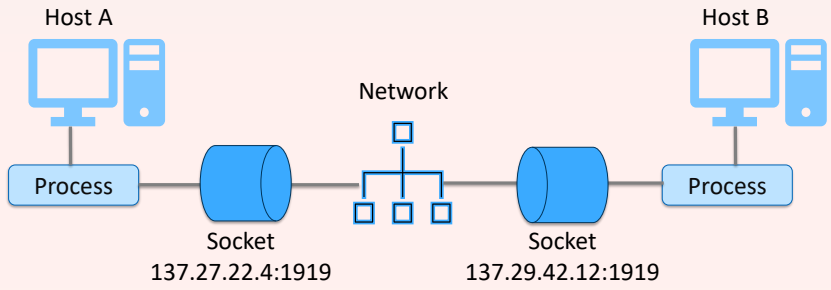
Practically, a socket is an IP address and a port number.

They mainly use TCP or UDP packets, giving us two basic types:

- Connectionless
- Connection-orientated

Ultimately, they act as an abstraction for the many complicated, low-level processes occurring to make data safely pass from one device to another.

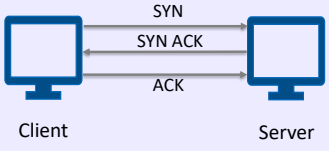
TCP/IP: Sockets



Distributed Computing: Sockets in java.net.*

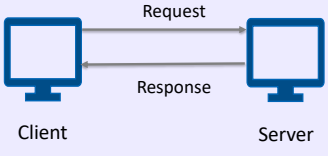
TCP:

- **`ServerSocket`** to listen for client requests.
- **`Socket`** to establishes a virtual connection.

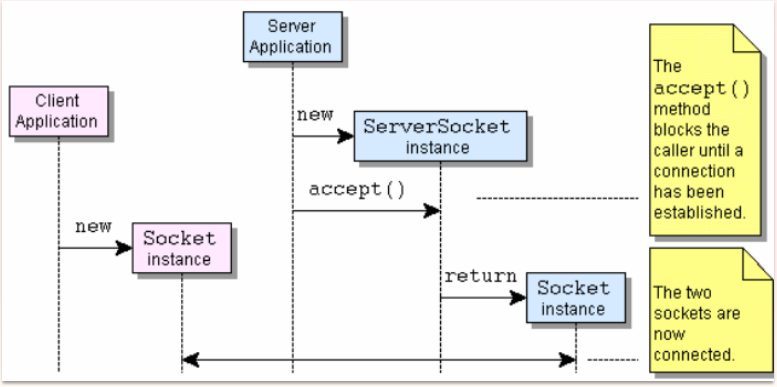


UDP:

- Both hosts communicate via a **`DatagramSocket`**.
- Carry messages using a **`DatagramPacket`**



Distributed Computing: TCP Sequence



Socket Programming: Socket and ServerSocket

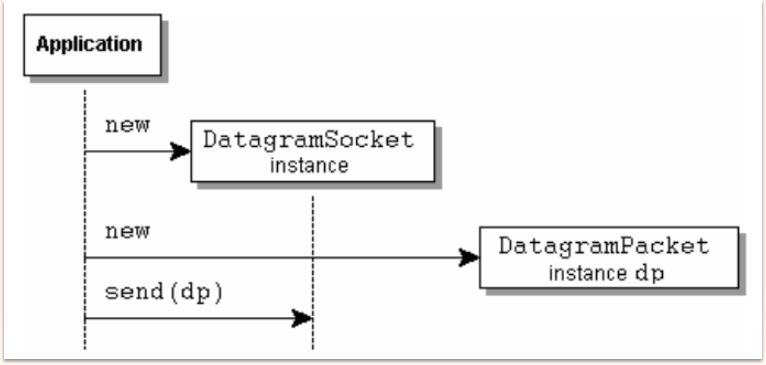
Client-side:

```
Socket clientSocket = new Socket("10.230.42.12", 4444);
   ...
```

Server-side:

```
SeverSocket serverSocket = new ServerSocket(4444);
Socket in = serverSocket.accept();
   ...
```

Distributed Computing: UDP Sequence



Socket Programming: DatagramPacket and DatagramSocket

Client-side:

```
byte[] buf = new byte[256];
DatagramSocket socket = new DatagramSocket();
DatagramPacket packet = new DatagramPacket(buf, buf.length,
                                           ipAdd, 2929);
socket.send(packet)
. . .
buf = new byte[256];
packet = new DatagramPacket(buf, buf.length);
socket.receive(packet);
```

Socket Programming: DatagramPacket and DatagramSocket

On the server side:

```
DatagramSocket socket = new DatagramSocket(4444);
 // Wait for client request

byte[] buf = new byte[256];
DatagramPacket packet = new DatagramPacket(buf, buf.length);
socket.receive(packet)

// Make a new buffer of data to send back
packet = new DatagramPacket(buf, buf.length, address, port);
socket.send(packet);
```

Concept Check: Data as a Sequence

Anyone notice the key difference between TCP and UDP sockets?



Socket Programming: TCP vs UDP

UDP:

- Connectionless
- Packet-orientated

That means data will be sent and received as an isolated chunk of bytes.

There is no expected order or sequence, only an allocated size.

TCP:

- Connection-orientated
- Sends a sliding windows of packets

So how does that change the programming?

Socket Programming: I/O Streams

Streams! Sequences of raw bytes make a stream of primitive data.

That, in turn, adds up to more complex data.

You've actually been using streams since your first year.

They've just been hidden under syntax:

`System.in`

`System.out`

`in` and `out` are just public attributes of that class.
Their data type?
`InputStream` and `PrintStream`

`System` is just a utility class that talks to your operating system.



Socket Programming: I/O Streams – Cont.

There are *many* different types of streams for different use cases.

To name a few:

AudioInputStream, ObjectInputStream, FileInputStream

PipedOutputStream, BytesArrayOutputStream, FilterOutputStream, ….

TCP Sockets allow us to send and receive data over a network.

Want to guess how that data arrives?

Yes, as I/O streams!



Socket Programming: I/O Stream Wrappers

Raw byte streams are great, but …

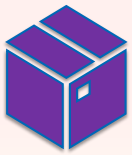
Working with the (input or output) can require a lot of manual processing.

This is where higher-level objects, known as wrappers become useful.

Up to this point, you've likely used the Scanner class for a similar I/O management in Java.

For example:

```
Scanner consoleIn = new Scanner(System.in);
int number = consoleIn.nextInt();
```



The Scanner is just one of the simplest ways to manage I/O streams.

Socket Programming: I/O Stream Readers and Writers - Client

Client Side:

```
Socket clientSocket = new Socket("127.0.0.1", 3939);
InputStream in = clientSocket.getInputStream();
OutputStream out = clientSocket.getOutputStream();
BufferedReader br = new BufferedReader(
                                new InputStreamReader(in)
                                );
PrintWriter pr = new PrintWriter(out);
pr.println("Sending this message to server");
pr.flush();
```

Socket Programming: I/O Stream Readers and Writers - Server

Server Side:

```
ServerSocket serverSocket = new ServerSocket(3939);
Socket clientSocket = serverSocket.accept();
InputStream in = clientSocket.getInputStream();
OutputStream out = clientSocket.getOutputStream();
BufferedReader br = new BufferedReader(
                                new InputStreamReader(in)
                                );
PrintWriter pr = new PrintWriter(out);
pr.println("Sending this back to the client");
```

DEMO

TCP and UDP Networking Applications using Java Sockets

Distributed Computing: Better Models

Socket based networking allows basic distributed exchange.

Not ideal for calling remote API's.

Additional code is needed to deal with versioning, protocol changes, marshalling and unmarshalling (converting objects to transferable data)

Help comes in the form of:

- Java RMI
- MSDCOM (Microsoft Distributed Component Object Model)
- CORBA (Common Object Request Broker Architecture).

While we will only briefly look at Java RMI, the others introduce some interesting ideas regarding platform independent communication between software.

Better Models: Java Remote Method Invocation

Designed to make communication between multiple Java programs, running in separate JVMs, as much like making a method call inside a single process as possible.

Uses a **client-side stub**:

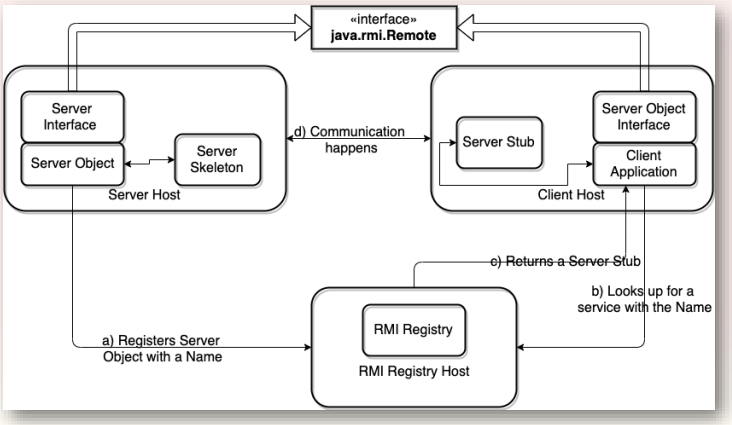
- an object in the client's JVM to represent the server-side object.

Uses a **server-side stub**:

- object maintaining network connections, marshalling and unmarshalling data on server side.

Central naming service called **rmiregistry:**
(runs on a central server and functions like a phone book, providing the server associated with a naming service).

Better Models: Java RMI Sequence

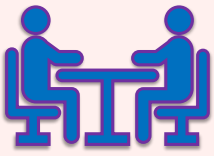


DEMO

Simple Java RMI Application

Summary: Part 2

- Sockets are a nice abstraction from low-level complexities of networking
- Java has a straightforward mechanism for programming sockets
- Using sockets for more complicated applications means a lot of (potentially) unnecessary work
- There have already been advancements in the way we program distributed systems



Lab: Socket programming in Java

Next lecture: The Web and Markup Languages

References and Resources

Networking the Web-Computing History:
https://www.computerhistory.org/timeline/networking-the-web/

Java Networking Tutorial:
https://docs.oracle.com/javase/tutorial/networking/overview/networking.html

Java RMI and CORBA:
https://www.oracle.com/technical-resources/articles/javase/rmi-corba.html

Thesis on World Wide Web:
https://www.w3.org/People/Frystyk/thesis/

Levels of Abstractions: Net, Web and Graph:
https://www.w3.org/DesignIssues/Abstractions.html

XML RPC:
http://1998.xmlrpc.com/